

Bash Scripting Basics

Unit 05 — Technical Foundations

A script is just the commands you already know — typed once and run on demand. Let's make the computer do the boring parts.

Learning objectives

By the end of this unit you can:

- **Explain** why automation matters in security.
- **Write** a runnable script with `#!/bin/bash`, `chmod +x`, run with `./script.sh`.
- **Use** variables, `read` input, and command substitution `$(...)`.
- **Use** positional arguments (`$1`, `$@`, `$#`).
- **Write** conditionals (`if`, `[ ]`) and `for`/`while` loops.
- **Define** a function and read an exit code (`$?`).
- **Build** a working **ping sweep** for the isolated lab subnet.
- **Apply** the rule: network scripts run **only** against the lab subnet.

Why automate?

- **Consistency** — same steps, every time, no missed checks.
- **Speed** — ping 254 hosts in seconds, not by hand.
- **Repeatability** — re-run it for a report; share it with a teammate.
- **Fewer mistakes** — write it carefully once.

A script is a **recipe**: a text file of commands the computer runs in order.

Anatomy of a script

```
#!/bin/bash  
# hello.sh – my first script  
echo "Hello from Bash!"
```

```
chmod +x hello.sh    # make it executable  
./hello.sh           # run it (the ./ means "in this folder")
```

- **Shebang** `#!/bin/bash` — tells the system which program runs the file.
- `#` starts a comment.
- `Permission denied`? You forgot `chmod +x`. `command not found`? You forgot `./`.

Variables & input

```
name="Jordan"           # NO spaces around the =  
echo "Hello, $name"  
echo "Hello, ${name}"   # braces are safer  
  
read -p "Target IP: " target  
echo "Working on $target"
```

🚨 The **#1 beginner bug**: `name = "x"` fails. It must be `name="x"`.

Quote your variables — use `"$name"` — to avoid surprises.

Command substitution

Capture a command's **output** into a variable with `$(...)`:

```
today=$(date)
echo "Date: $today"

myip=$(hostname -I)
echo "This machine: $myip"
```

- `$(date)` runs `date` and hands back its output.
- The variable holds the **result**, not the command.

Arguments make scripts flexible

Pass info *to* a script instead of hard-coding it:

```
./pingone.sh 192.168.56.1
```

Slot	Holds
<code>\$1</code> , <code>\$2</code>	first, second argument
<code>\$@</code>	all the arguments
<code>\$#</code>	how many arguments

Pass the IP **to** the script — don't bake one IP inside it.

Conditionals & exit codes

```
if [ $# -eq 0 ]; then
    echo "Usage: ./pingone.sh <ip>"
    exit 1          # non-zero = problem
fi
```

- `if` / `then` / `else` / `fi` — run code only when a test is true.
- `[ ]` is the **test**: `-eq`, `-lt`, `-gt`, `-eq 0` ...
- Every command returns an **exit code**: `0` = success, else = problem.
- Read it with `$?` — *right after* the command runs.

Ping one host (using `$?`)

```
target=$1
ping -c 1 -W 1 "$target" > /dev/null 2>&1

if [ $? -eq 0 ]; then
    echo "$target is UP"
else
    echo "$target is DOWN"
fi
```

- `-c 1` = send one ping · `-W 1` = wait at most 1 second (don't hang).
- `> /dev/null 2>&1` = throw away ping's own output.
- `ping` returns `0` when the host **replied**.

Loops & functions

```
for host in $(seq 1 254); do
    echo "192.168.56.$host"
done
```

```
check_host() {                # a reusable named block
    ping -c 1 -W 1 "$1" > /dev/null 2>&1
    if [ $? -eq 0 ]; then
        echo "$1 is UP"
    fi
}
```

10

Putting it together: the ping sweep

```
#!/bin/bash
# sweep.sh — ping sweep of a lab /24. ISOLATED LAB SUBNET ONLY.
subnet="192.168.56"      # change only to your assigned lab subnet

check_host() {
    ping -c 1 -W 1 "$1" > /dev/null 2>&1
    if [ $? -eq 0 ]; then echo "$1 is UP"; fi
}

echo "Sweeping $subnet.0/24 (lab only) ..."
for host in $(seq 1 254); do
    check_host "$subnet.$host"
done
```

Ethics & Authorization

A script does not change the law.

A one-line loop pings all 254 addresses in seconds. **Automation just lets you break the rules faster.**

Scanning a network you don't own or have written permission to test can violate the **CFAA** and state law.

The only legal target this week: the isolated host-only lab subnet. Confirm with `ip addr` *before every* network command.

Key vocabulary

Term	Meaning	
Script / Shebang	File of commands run in order / the <code>#!/bin/bash</code> first line	
Executable	A file allowed to run; set with <code>chmod +x</code>	
Variable	A named box holding a value	
Command substitution	Capturing a command's output: <code>\$(command)</code>	
Positional parameter	<code>\$1</code> , <code>\$2</code> , <code>\$@</code> , <code>\$#</code>	
Conditional / Loop	<code>if</code> ... <code>fi</code> / <code>for</code> , <code>while</code>	
Function	A named, reusable block of code	13
Exit code (<code>\$?</code>)	<code>0</code> = success, anything else = problem	Curriculum by Al Hammond — PNPT, CRT0, OSCP, BSCP

Lab launch: build a ping sweep

Platform: your **Kali VM** on the **host-only / isolated** lab network.

First, every day:

```
ip addr          # confirm you're on 192.168.56.x (lab subnet)
mkdir -p ~/unit05 && cd ~/unit05
```

You'll build three scripts:

1. **banner.sh** — recon banner (variables, `read`, `$(date)`)
2. **pingone.sh** — ping one host, decide UP/DOWN by exit code
3. **sweep.sh** — loop + function + conditional = full /24 sweep

14

Recap

- A script = your commands in a file: **shebang**, `chmod +x`, `./run.sh`.
- **Variables** (no space around `=`), `read`, and `${...}` substitution.
- **Arguments** `$1` / `$#` make scripts flexible.
- `if [ ]` + **exit code** `$?` make decisions; `0` = success.
- **Loops** + **functions** build the ping sweep over the /24.
- Automation is power — and authorization matters *more*, not less.

Exit ticket & discussion

1. What two commands turn a text file into a runnable program?
2. After a command runs, where is its exit code stored — and what means success?
3. Write a `for` line that loops the numbers 1 through 254.

Discuss: Name one task you could now automate — and one place it would be **illegal** to run this script.

Submit your three scripts, a screenshot of each running, and your list of live lab hosts.